Engineering reliable agentic AI systems

Session 3. Research Loops and Model Context Protocol (MCP).

Saturday 29 August 2026. 12:35 Central. Forty minutes.

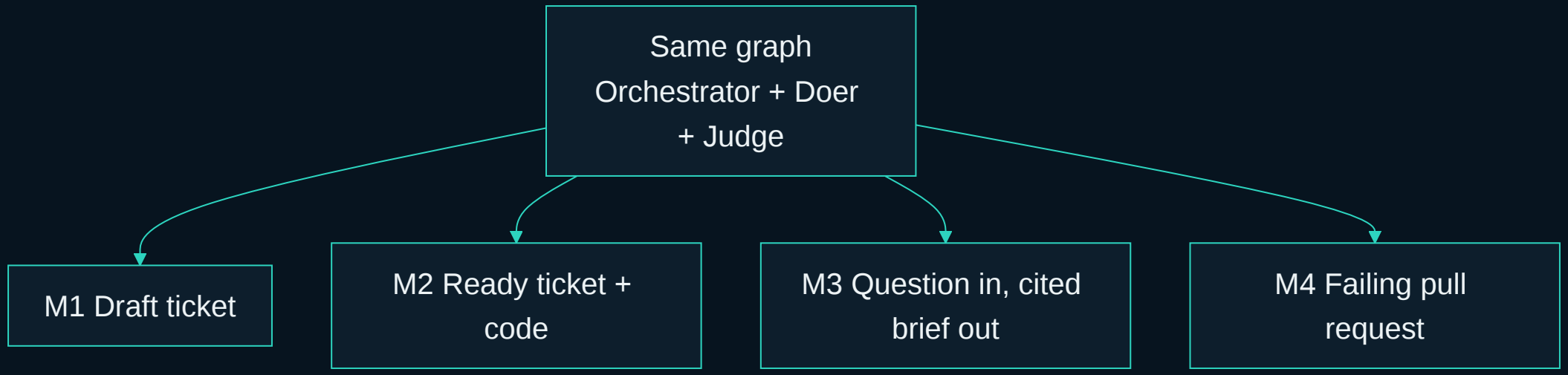
Rick Hightower. Spillwave. Packt workshop.

You already have the graph. This hour points it at a question.

BLOCK	START	MINUTES
Module 3 research + MCP	12:35	40
Talk. Tool contracts	12:35	10
Lab. One research assistant	12:45	25
Retries, budgets, failure	13:10	5
Break	13:15	15

Artifact they keep: one working research assistant. Not a survey.

Same graph. Four objects. This hour is the third.



`slides/diagrams/mermaid/s3-same-graph.mmd`

Same graph. New object.

Orchestrator, doer, judge. The exact three parts from Module 1.

The object is a question. The artifact is a cited brief.

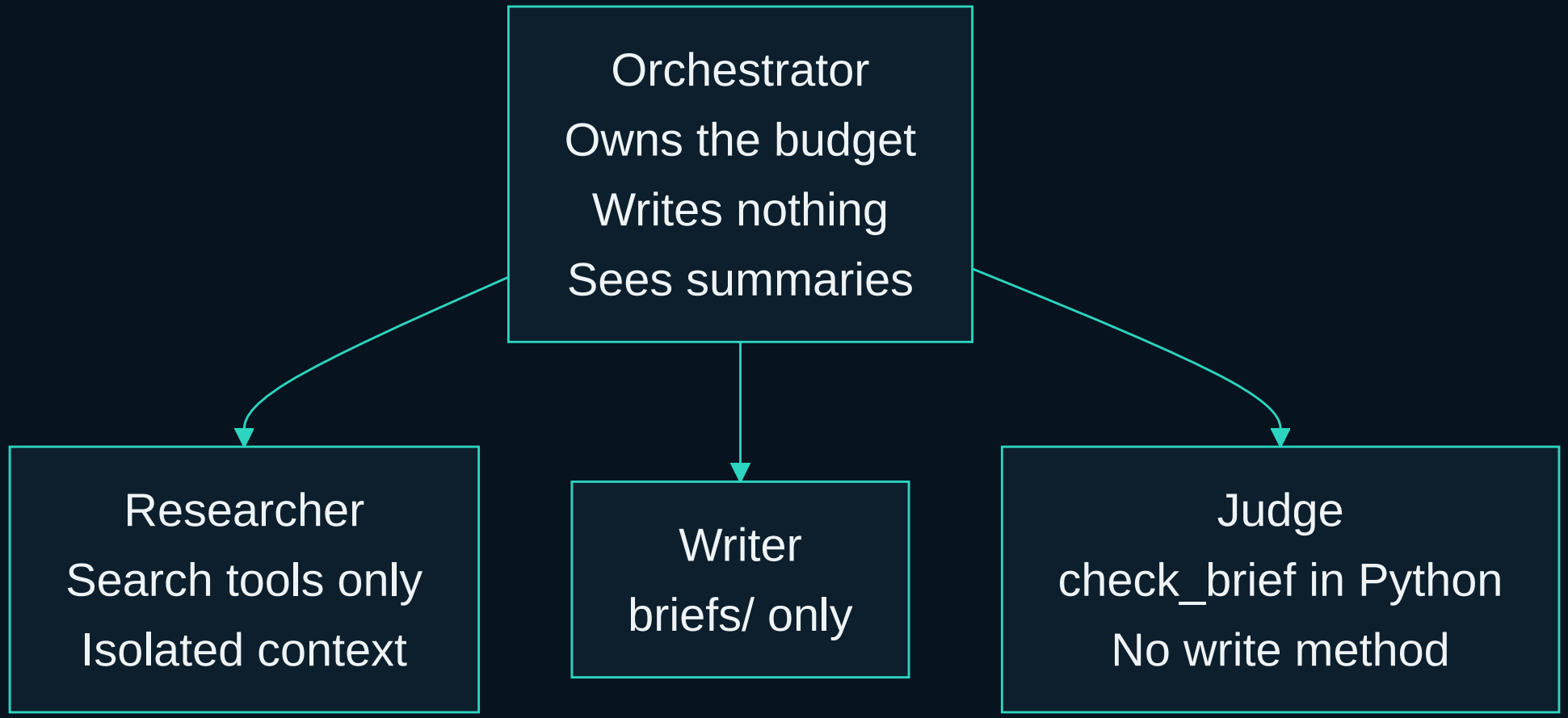
The judge still holds no write path.

The only new thing is a tool that reaches outside the machine.

Research is a subagent

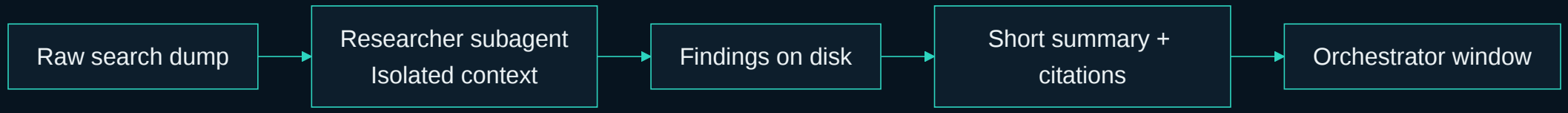
So the orchestrator window stays clean.

Four roles. The orchestrator never sees the dump.



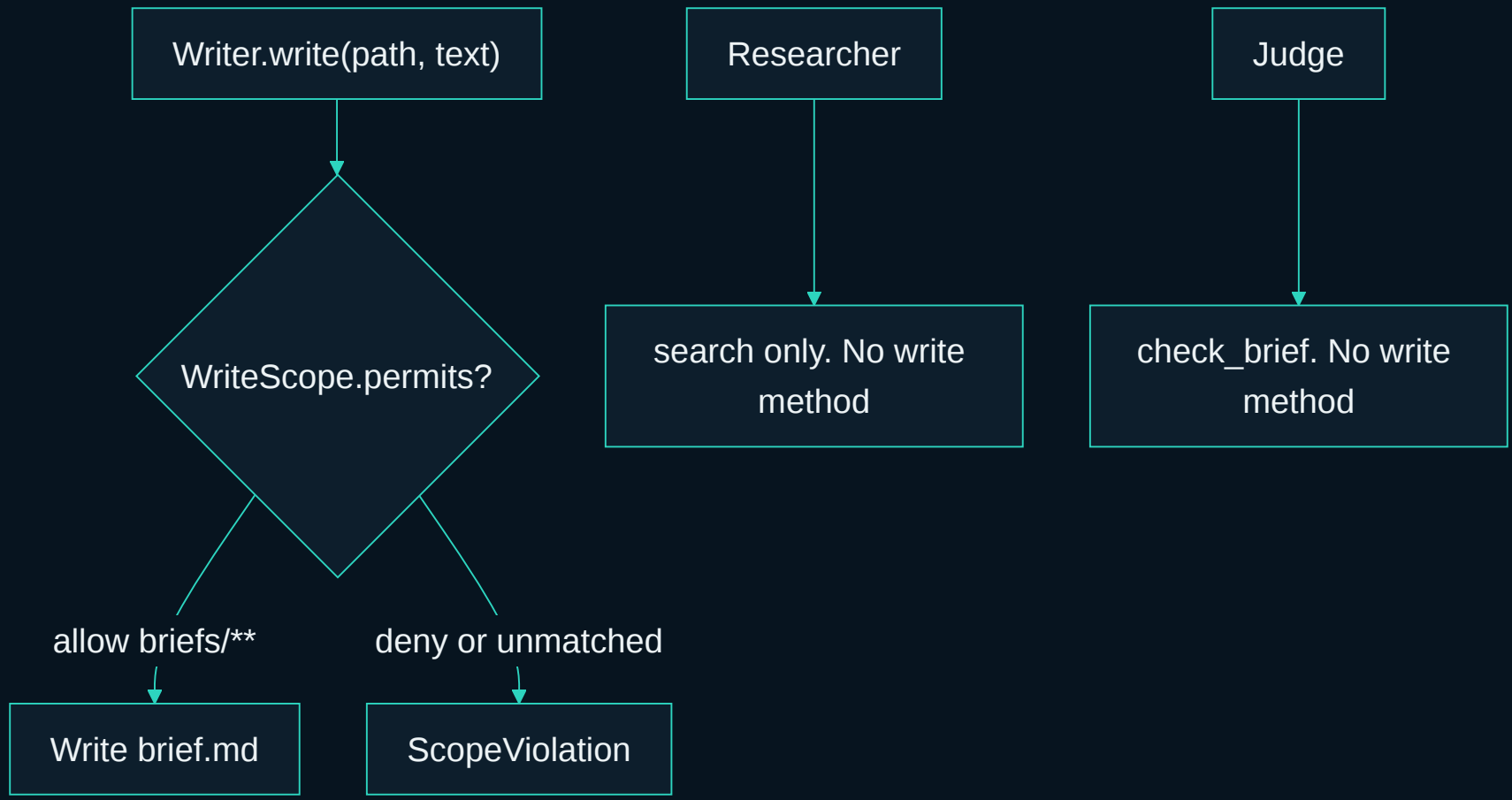
`solutions/sol3_research_deep_agents/roleplan.py` · loop `research`

Isolated context. A summary comes back. The dump does not.



Raw search never returns to the orchestrator. A summary does.

Writer writes the brief. Nobody else does.

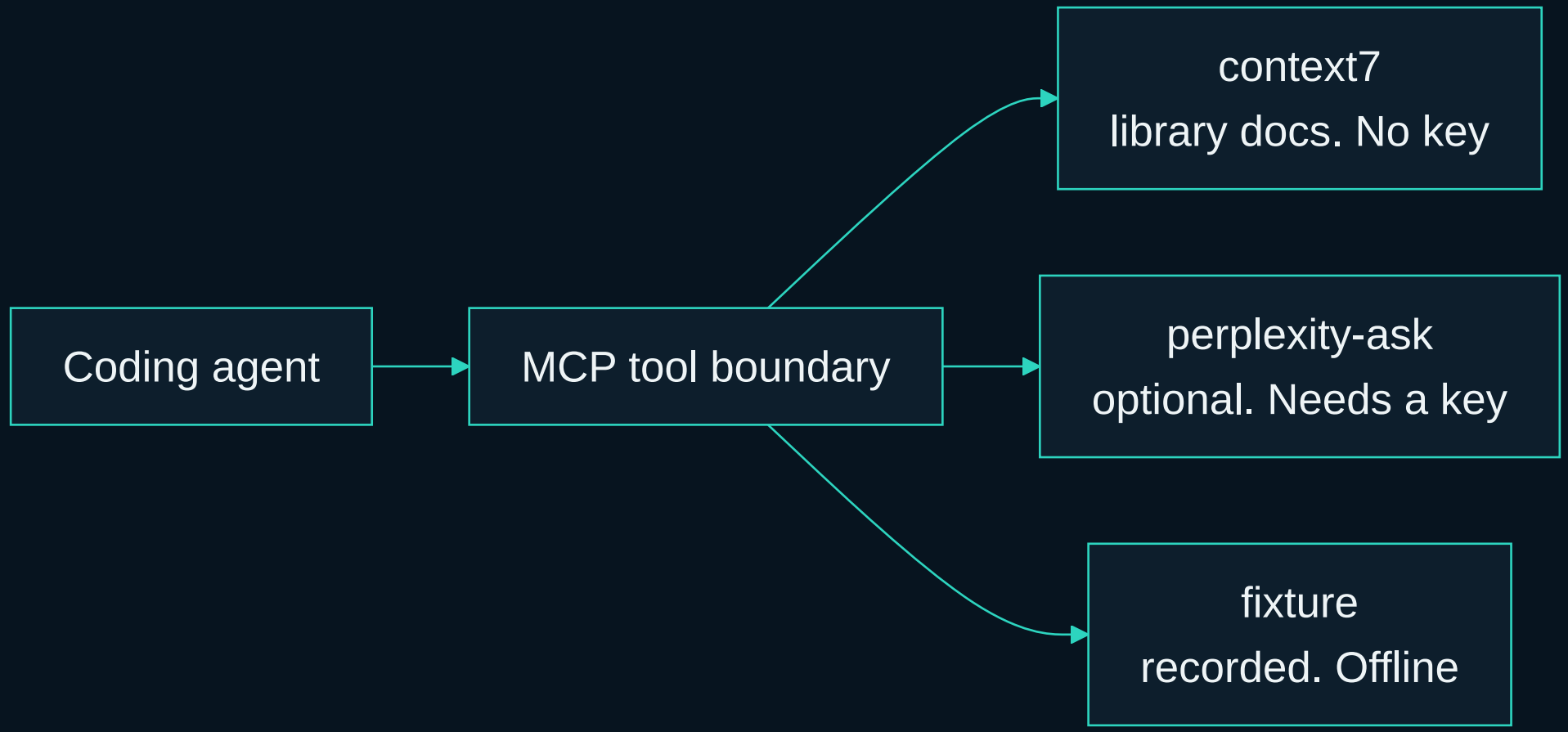


Citations are arithmetic. The judge does not get a vote.

A safe tool boundary

Narrow. Read-only. Named.

Model Context Protocol is how the agent reaches outside itself.

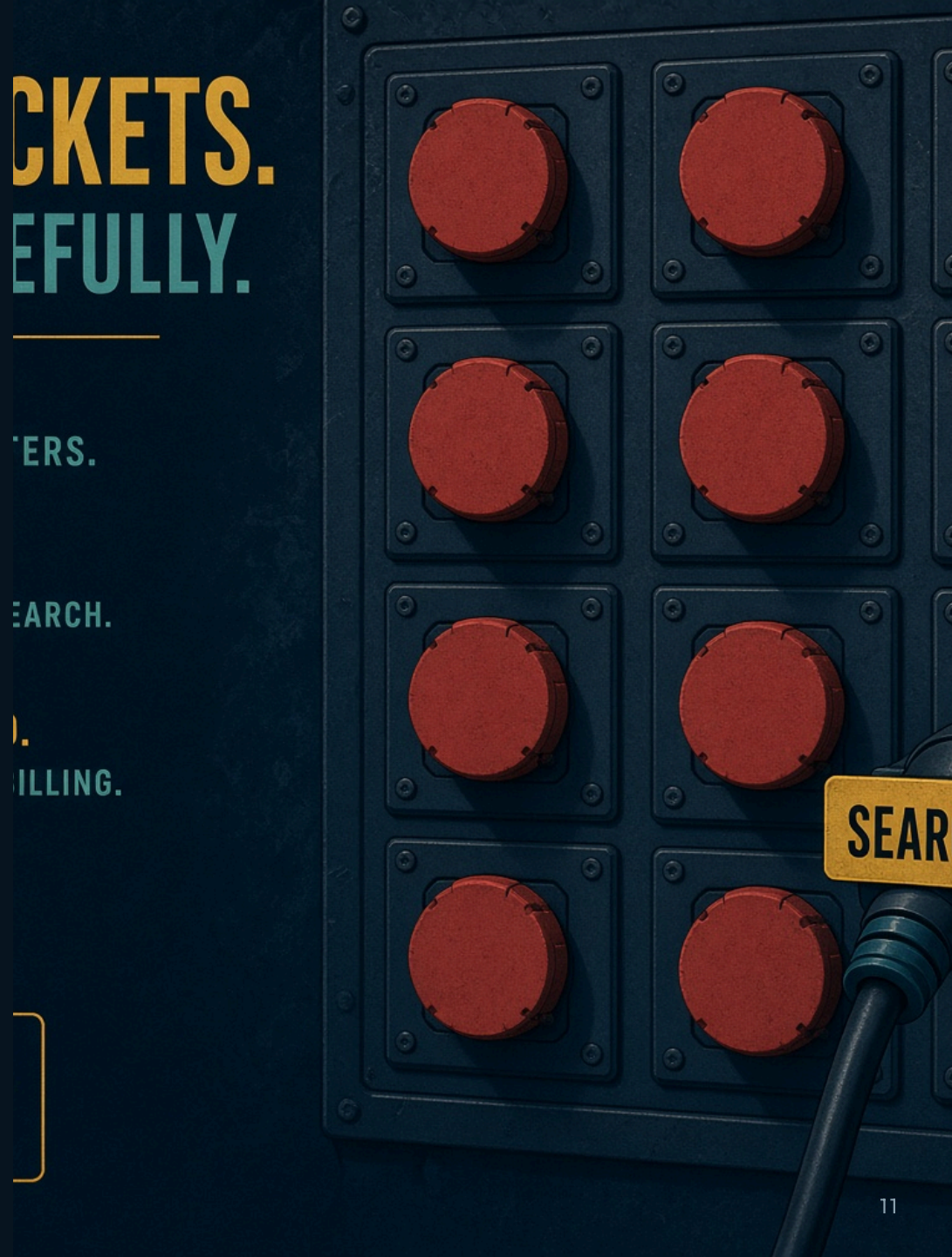


`.mcp.json` ships with this repo. Approve `context7` at minimum.

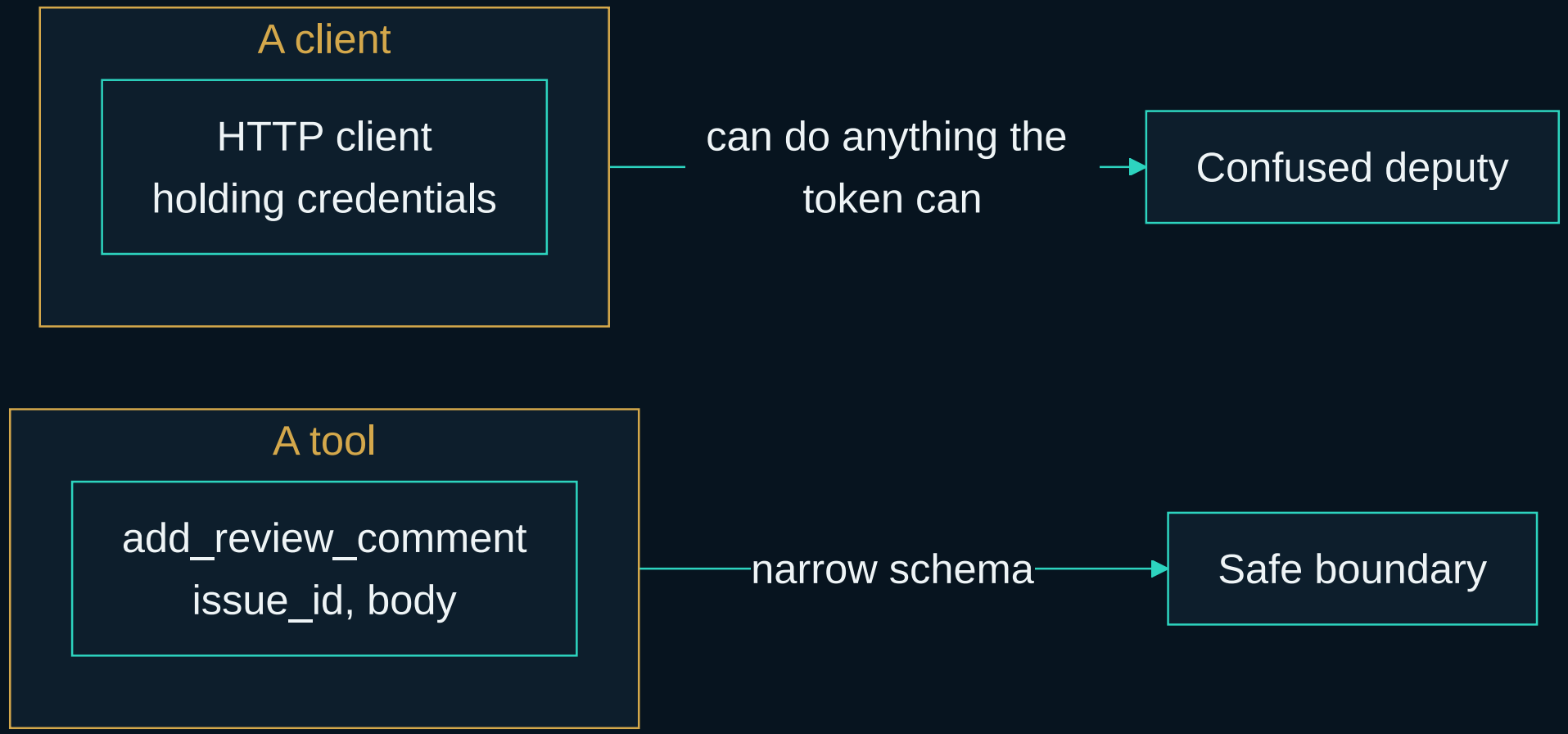
Nothing in the labs requires `perplexity-ask`.

A safe tool boundary is narrow, and it is read-only.

- Allowed: search, and write into this loop's own output folder.
- Denied: merge, deploy, ticket state, anything in production.
- A tool contract is a short list of what an agent may do.
- The interesting list is what it may not.



A narrow schema beats a broad one.



`add_review_comment(issue_id, body)` is a tool.

A Hypertext Transfer Protocol (HTTP) client holding your credentials is a liability.

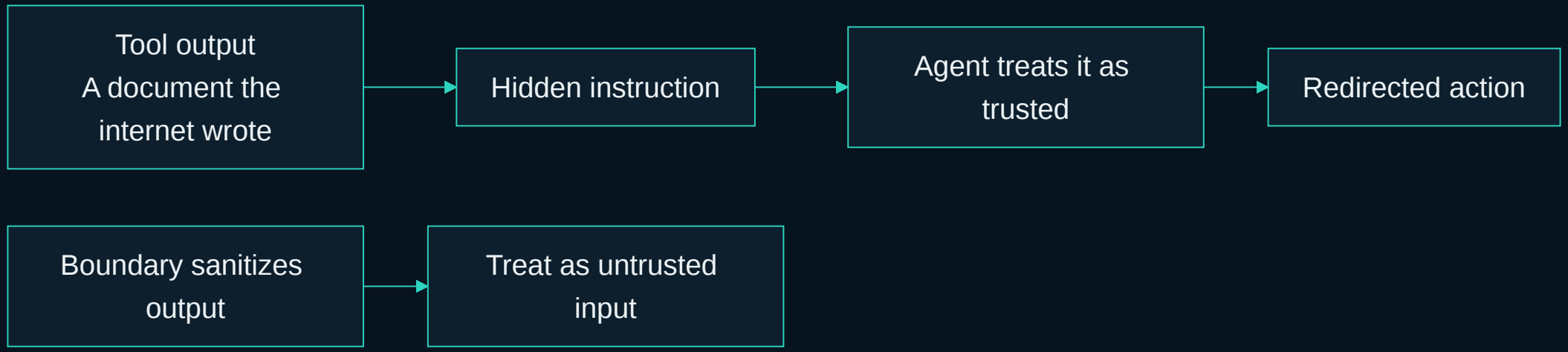
Agents reach for the bigger tool. This is measured.

- Mainstream agents often chose a higher-privilege tool when a lower one was enough.
- Transient failures made that escalation more likely, not less.
- Prompt-based controls gave only limited mitigation.

You do not fix this with a stronger sentence. You fix it by not shipping the sledgehammer.

ToolPrivBench, 2026. Yang et al. arXiv:2606.20023

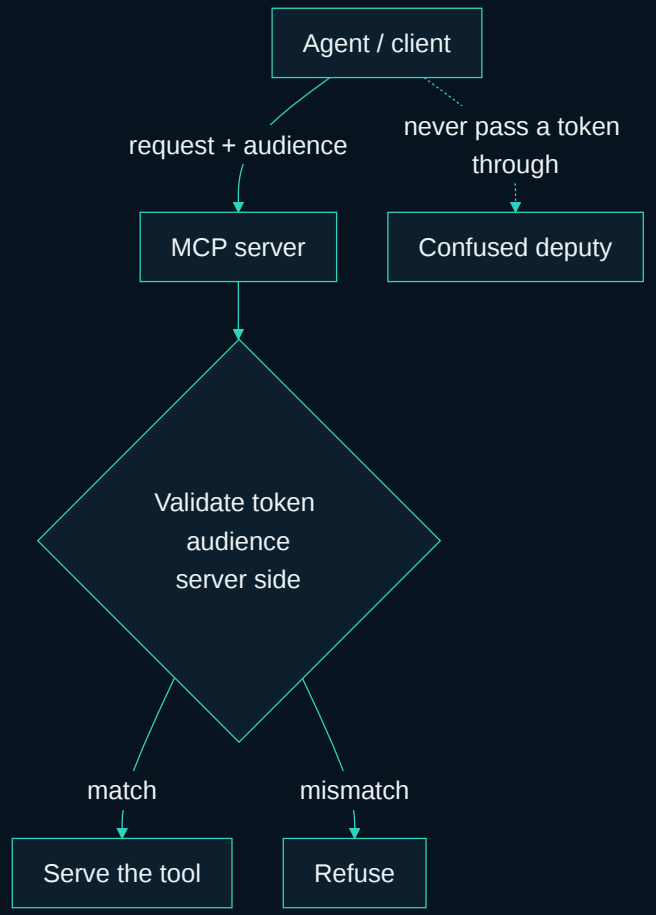
What comes back from a tool is untrusted input.



AgentDojo showed that tool output can carry instructions, and that those instructions can redirect the agent.

Your search results are a document the internet wrote, not a system prompt.

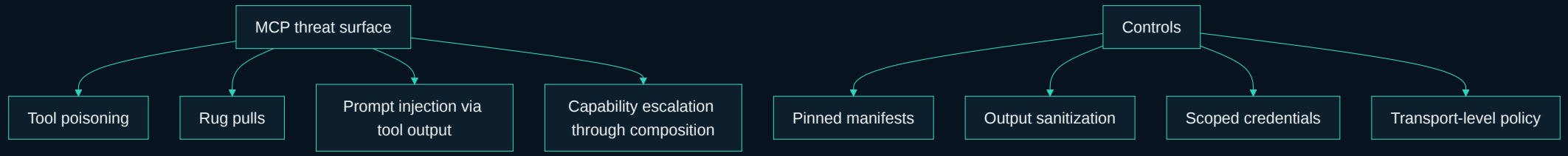
Authorization is a property of the tool boundary.



The MCP authorization spec makes the same call.

Validate the token audience server side. Never pass a token through.

MCP has a threat surface. Name it, then pin it.



A prompt is not a control. A pinned manifest is.

Threats: tool poisoning, rug pulls, prompt injection via tool output, capability escalation through composition. Controls: pinned manifests, output sanitization, scoped credentials, transport-level policy.

One boundary. Three backends. You pick.

BACKEND	WHEN
Perplexity over MCP	You set `PERPLEXITY_API_KEY`
Your agent's own WebSearch	No key, but the tool is there
A recorded fixture	Offline, or the wifi in this room

The loop calls one function. It never learns which one answered.

The clock. Ten minutes of talk. Twenty-five of typing.

- Fill `loop.py`. Two functions. Nothing else.
- The question is boring on purpose.
- Stuck? Stop typing and watch.
- Copy `solutions/sol3_research/loop.py` and you continue with a working artifact.

Nobody leaves this room behind.

You should be at 10 minutes here.

Lab 3. The research assistant

25 minutes. A question in. A cited brief out.

Lab 3. The research assistant. 25 minutes.

```
cd labs/lab3_research
claude -p "$(cat prompts/claude-code.md)"      # or codex, grok, opencode

task loop:research -- --question "sqlalchemy nullable datetime column" \
  --backend fixture
```

Fill `loop.py`. Two functions: `plan_questions` and `check_brief`.

Falling behind is fine: copy `loop.py` from `solutions/sol3_research/`.

Fill one file. Two functions. The backend does not appear.

```python
def plan_questions(question: str) -> list[str]:
    raise NotImplementedError("fill me in")

def check_brief(body: str, sources: list[str]) -> brief.BriefScore:
    raise NotImplementedError("fill me in")
```

A plan step you cannot check is a wish.

The loop calls one function and never learns whether Perplexity, WebSearch, or a fixture answered.

`labs/lab3_research/loop.py`

`plan_questions` *is a template you can check.*

```python
def plan_questions(question: str) -> list[str]:
    return [
        question,
        f"{question} common mistake",
        f"{question} how to verify",
    ]
```

Each sub-question is one you can tell was answered or not.

Swapping in a model here is the lab's stretch goal. The checks downstream do not change when you do.

`loops/researcher.py`

`check_brief` *does not ask a model.*

```python
def check_brief(body: str, sources: list[str]) -> brief.BriefScore:
    return brief.check(body, sources)
```

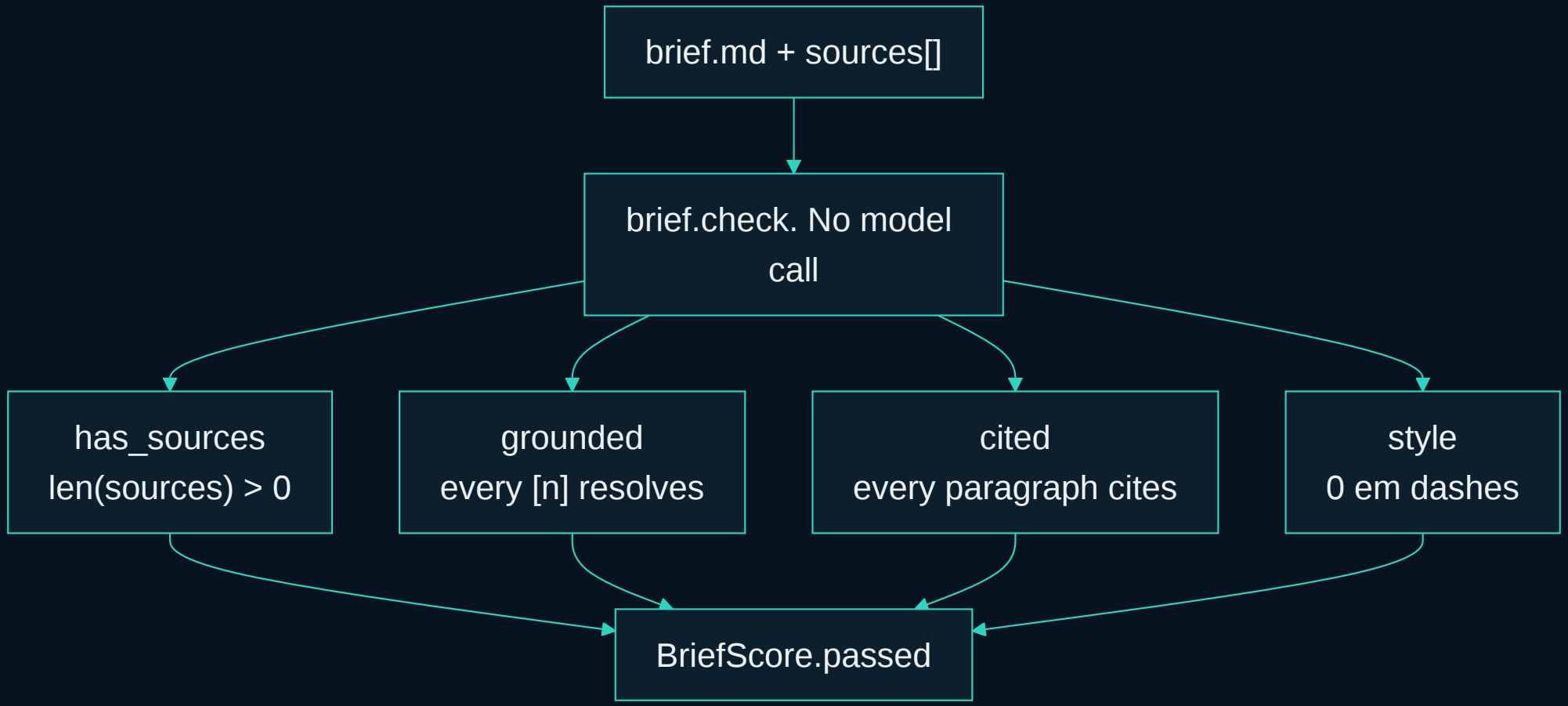
Two things are arithmetic, not judgement.

`grounded` : every citation marker resolves to a source actually retrieved.

`cited` : every claim paragraph carries a citation.

`solutions/sol3_research/loop.py`

The judge reads the brief. It does not read it thoughtfully.



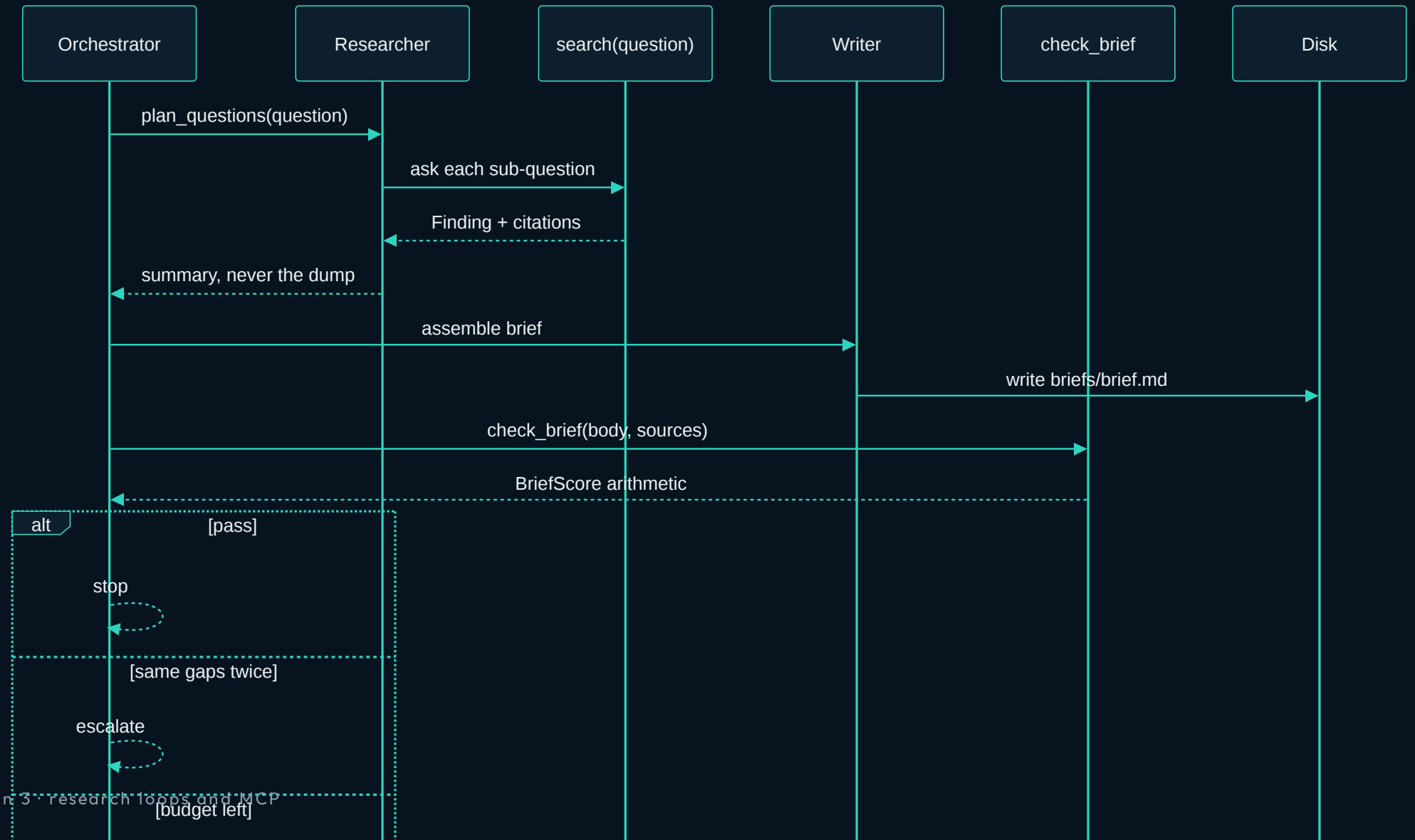
A confident sentence nobody can trace is the failure that matters.

Two functions in `loops/brief.py`. Both refuse to argue.

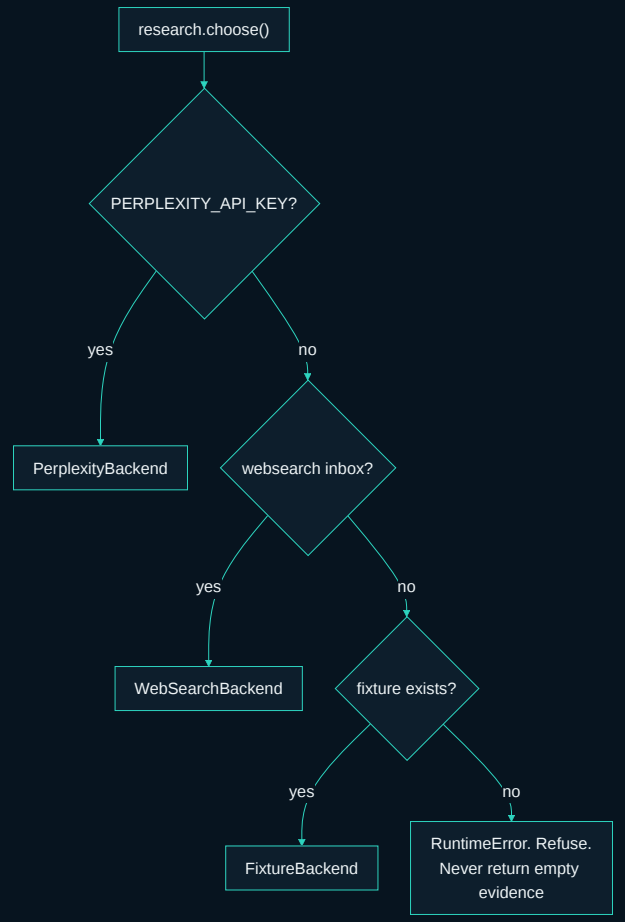
`ungrounded_citations` returns markers that point at a source which was never retrieved.

`strip_em_dashes` replaces them deterministically. Code spans are left alone.

One live loop. Question in. Cited brief out.



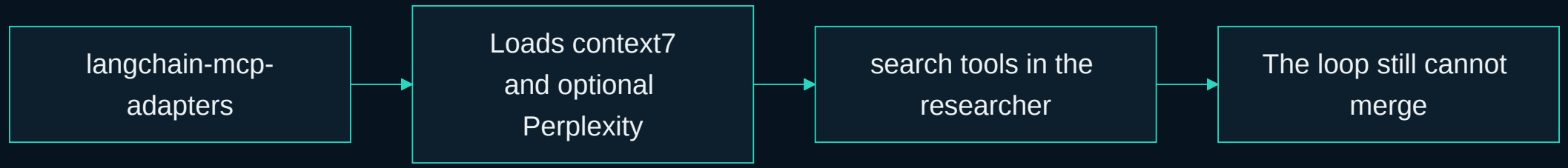
`research.choose` *picks a backend. The loop stays ignorant.*



Saturday path is `--backend fixture`. `loops/fixtures/research.json`.

`loops/research.py`

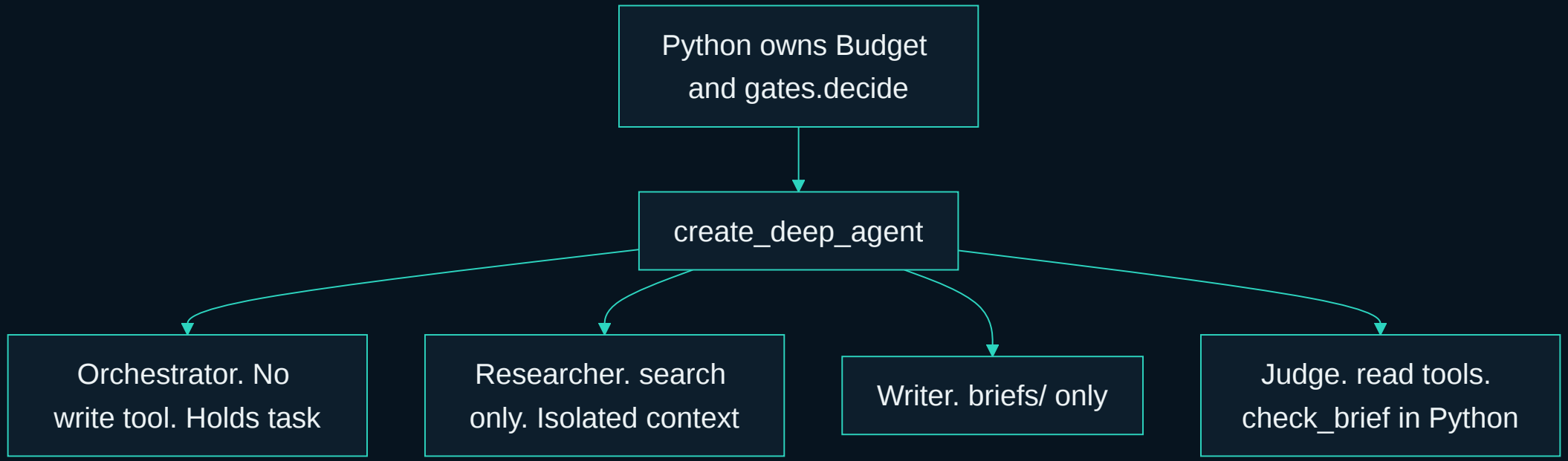
`langchain-mcp-adapters` *loads the servers. The loop still cannot merge.*



Loading a server is not granting production. The wall is the tool list.

The researcher gets search. The writer gets `briefs/`. The judge gets `check_brief`.

Saturday is two functions. The takehome is Deep Agents.



Takehome: `solutions/sol3_research_deep_agents/` . Issue #119.

LangChain Deep Agents ships a research agent as the default example.

Falling behind is fine. Copy the answer and keep going.

```
cp loop.py loop.py.my-attempt
cp ../../solutions/sol3_research/loop.py .
```

You now have a working research assistant.

Read `solutions/sol3_research/SPEC.md` later if you want the step by step.

Nobody is graded here.

Read the gate. Grounded and cited are arithmetic.

```
PASS  has_sources    2 sources retrieved
PASS  grounded       every citation resolves
PASS  cited          every paragraph cites a source
PASS  style          0 em dashes

backend: fixture   budget: $0.00 / $0.20 (soft $0.10), 3/8 calls
gate:    pass
```

A confident sentence nobody can trace is the failure that matters.

`loops/brief.py` · `check()`

Walk the room. Point at the brief, not at the model.

Work from `labs/lab3_research`. Fill only `loop.py`.

If you stall, read `loops/researcher.py`, `loops/research.py`, and `loops/brief.py`. That is the answer, not a hint.

Three exits. No fourth.

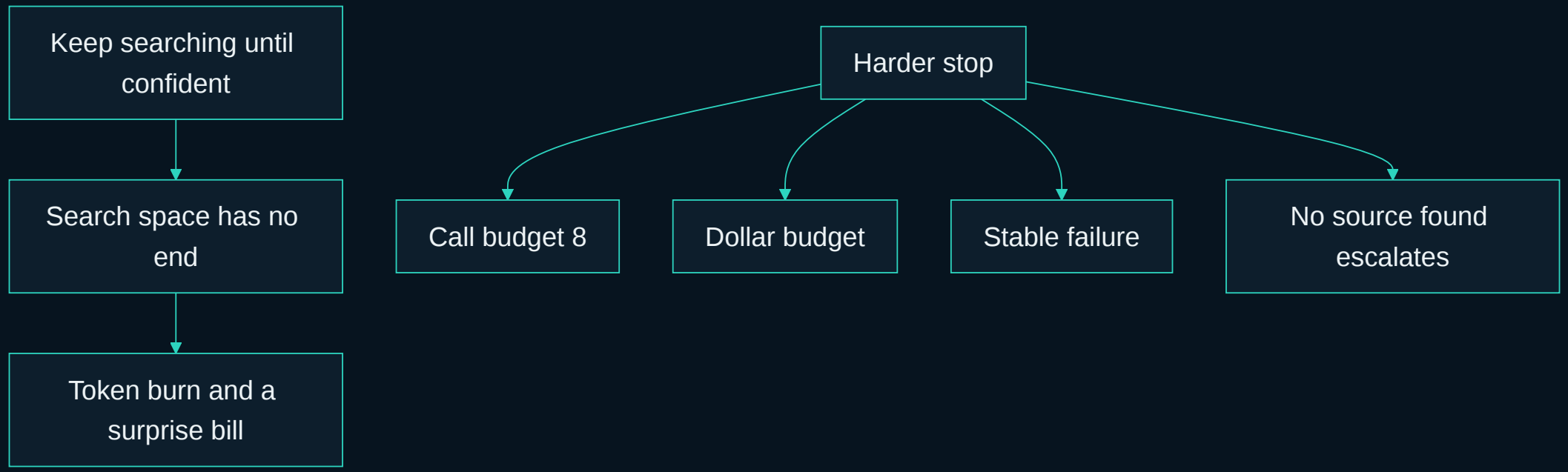
1. The brief is grounded and clean.
2. The search budget is spent.
3. No source could be found, which escalates rather than shipping an uncited brief.

You should be at 35 minutes here.

Retries, budgets, failure modes

A research loop needs a harder stop than code does.

Keep searching until confident is not a stop condition.



The search space has no end, so the loop has to be told where the end is.

The budget is a type. The hard cap raises.

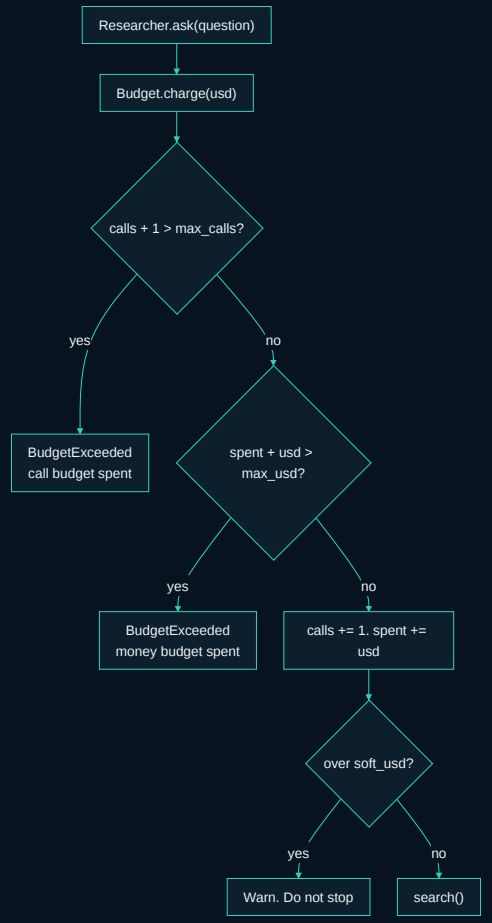
```python
@dataclass
class Budget:
    max_usd: float = 1.0
    max_calls: int = 8
    soft_usd: float | None = None

    def charge(self, usd: float) -> None:
        if self.calls + 1 > self.max_calls:
            raise BudgetExceeded(f"call budget spent: {self.max_calls} calls")
        if self.spent_usd + usd > self.max_usd:
            raise BudgetExceeded("money budget spent")
```

A budget that only warns is a budget that gets ignored at three in the morning.

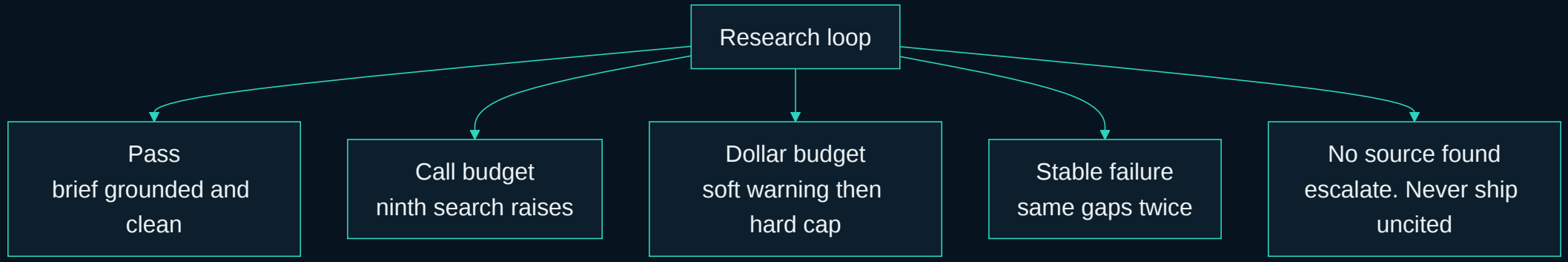
`loops/research.py` · `Budget` , `BudgetExceeded`

Soft warns. Hard raises. The ninth search does not run.



Live loop: `Budget(max_usd=0.20, max_calls=8, soft_usd=0.10)`.

Four stops. The forgotten one is still stable failure.



Call budget 8. Dollar budget. Stable failure. No-source escalates.

Python holds the loop, so the model never counts its own retries.

Two numbers worth remembering.

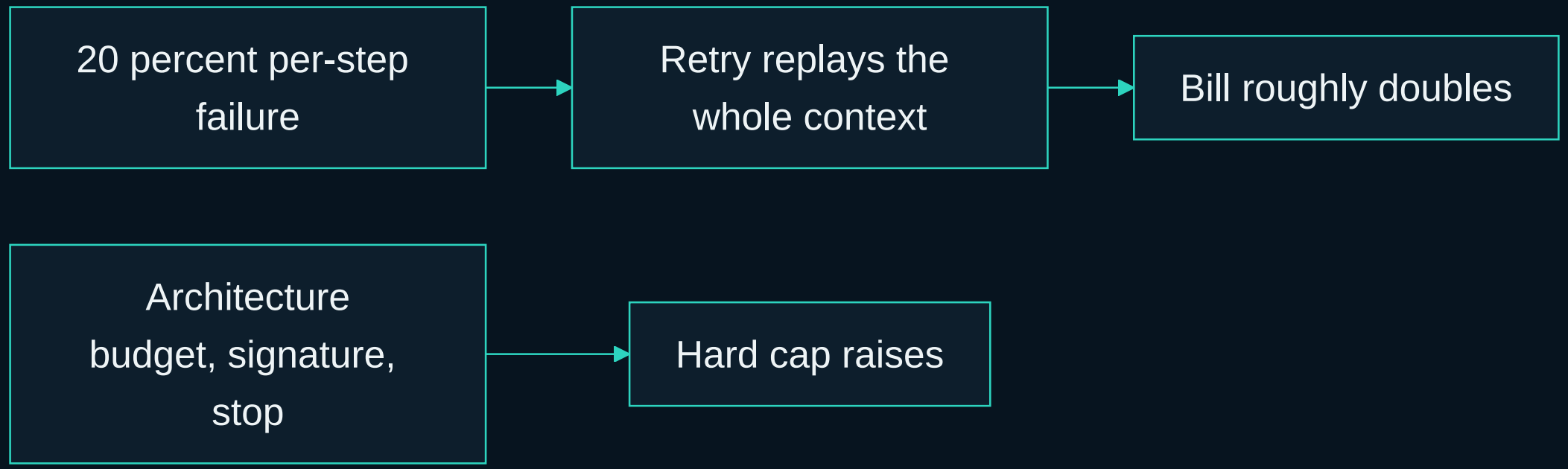
NUMBER	WHAT IT SAYS
15.7%	Share of recorded agent failures that are one step, repeated
12.4%	Share where the agent did not know it was already done

The same gaps twice is not progress. Stopping is the feature.

`signature` is what failed, not how it was worded.

`loops/gates.py` · `decide()`

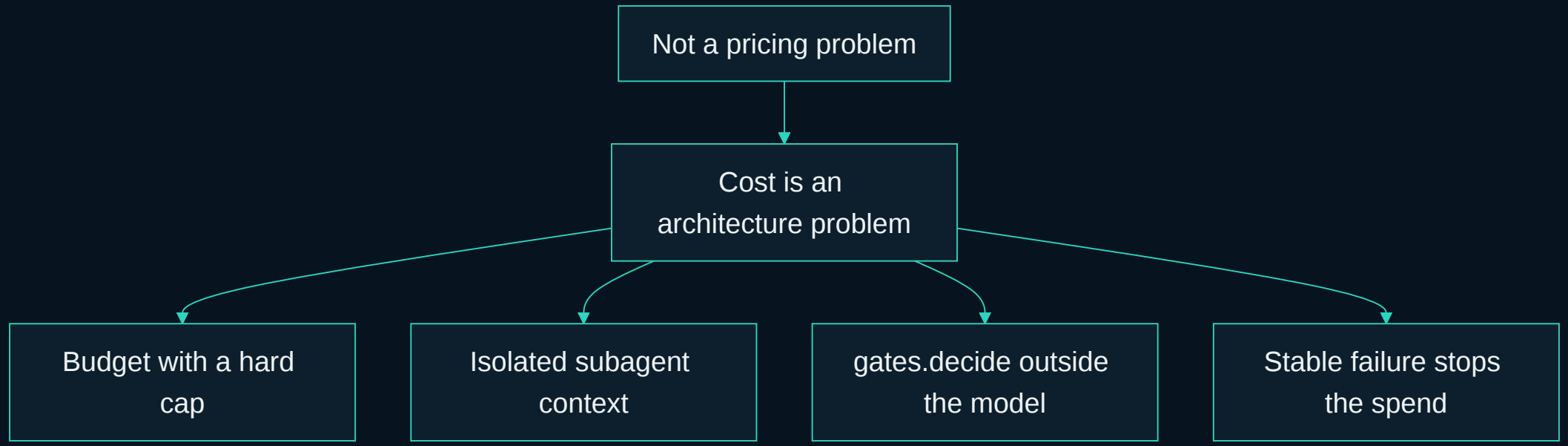
A twenty percent miss can roughly double the bill.



Retries are not linear. A retry usually replays the whole context.

That is why the researcher is a subagent. The orchestrator window stays small.

Cost is an architecture problem, not a pricing problem.



Cheaper tokens do not fix a loop that cannot stop.

Six lines to keep.

1. A safe tool boundary is narrow and read-only.
2. Tool output is untrusted input.
3. One function, three backends. The loop never learns which.
4. Citations are arithmetic. `check_brief` does not guess.
5. Call budget, dollar budget, stable failure, no-source escalate.
6. Cost is an architecture problem, not a pricing problem.

Primary references for this session.

- Yang et al. ToolPrivBench. 2026. arXiv:2606.20023. OpenReview AXH6buTOVx
- AgentDojo. Tool output is untrusted input that can carry instructions
- MCP authorization spec. Validate token audience server side. Never pass a token through
- `loops/research.py`, `loops/brief.py`, `loops/researcher.py`, `loops/gates.py`
- `labs/lab3_research/ARCHITECTURE.md`, `MCP.md`
- `solutions/sol3_research_deep_agents/` · Issue #119

Break. 15 minutes.

Next: the same stack, with nobody at the keyboard.

Module 4. Production architecture. A failing pull request in. A mergeable branch, or an honest explanation.